

“Actividad 3.3”

Integrantes:	German Antonio Gonzalez Rodriguez
Profesor:	Dra. Genoveva Viridiana Silva Rodríguez
Semestre	2025-2026/II

BrazzaHeader.vue — Banner principal

Propósito: Muestra el logotipo del restaurante junto con el nombre del sistema y la dirección. Es un componente de solo presentación: no modifica ningún dato, simplemente recibe valores y los despliega.

Cómo funciona el Template:

Código / Elemento	Explicación
<code><header class="header"></code>	Elemento semántico HTML5 que actúa como contenedor azul flex.
<code>:src="logo"</code>	Enlace dinámico al prop logo. El : (dos puntos) indica que es una expresión Vue, no texto fijo.
<code>{{ title }}</code>	Interpolación de texto. Vue sustituye esto con el valor del prop title en tiempo real.
<code>{{ address }}</code>	Lo mismo para la dirección del restaurante.

Cómo funciona el Script:

Código	Explicación
<code>props: { logo, title, address }</code>	Declara los tres datos que el componente espera recibir de App.vue. Si no se pasan, se usan los valores default.

Cómo funciona el CSS:

Regla CSS	Explicación
<code>.header { display: flex }</code>	Alinea el logo y el texto en fila horizontal.
<code>@media (max-width: 900px)</code>	En tablet, reduce el tamaño del logo y las fuentes.
<code>@media (max-width: 480px)</code>	En móvil, cambia a flex-direction: column para que el logo quede encima del texto.

InputField.vue — Campo de texto con label flotante

Propósito: Campo de texto reutilizable con etiqueta que flota sobre el borde. Con variantes de color y un botón opcional '?' que abre un modal de ayuda.

Cómo funciona el Template:

Código / Elemento	Explicación
<code>position: relative</code> en <code>.input-wrap</code>	Permite que el label flotante se posicione sobre el borde del input usando <code>position: absolute</code> .
<code>v-if="help"</code>	El botón '?' solo aparece en el DOM si se pasó un valor al prop <code>help</code> . Si <code>help</code> está vacío, el botón no existe.
<code>:class="`input-wrap__help--\${color}`"</code>	Agrega dinámicamente la clase de color (blue, green o yellow) según el prop <code>color</code> .
<code>@click="\$emit('open-modal', help)"</code>	Al hacer clic en '?', emite el evento <code>open-modal</code> y pasa el ID del modal como dato.
<code>:value="modelValue"</code>	Parte 1 de v-model manual: muestra el valor actual en el input.
<code>@input="\$emit('update:modelValue', ...)"</code>	Parte 2 de v-model: cada vez que el usuario escribe, emite el nuevo valor hacia <code>App.vue</code> .

Cómo funciona el Script:

Código	Explicación
<code>modelValue</code>	Prop estándar de Vue 3 para v-model. <code>App.vue</code> escribe aquí y el componente lo muestra.
<code>emits: ['update:modelValue', 'open-modal']</code>	Declara formalmente los eventos que puede emitir. Mejora el rendimiento y la documentación automática.

Cómo funciona el CSS:

Regla CSS	Explicación
<code>.input-wrap__label { position: absolute }</code>	Coloca la etiqueta justo en el borde superior del input, creando el efecto flotante.
<code>background: var(--color-bg)</code> en el label	El fondo del label coincide con el fondo de la página, por eso 'corta' el borde del input visualmente.
<code>pointer-events: none</code>	El label no intercepta clics — los clics pasan al input que está debajo.
<code>.input--blue / .input--green / .input--yellow</code>	Tres variantes de color aplicadas dinámicamente via <code>:class</code> según el prop <code>color</code> .

ActionButton.vue — Botón de acción principal

Propósito: Botón con forma de píldora y fondo verde oscuro. Implementa el estado disabled que lo deshabilita visual y funcionalmente cuando el formulario está incompleto.

Cómo funciona el Template:

Código / Elemento	Explicación
<code>:disabled="disabled"</code>	Enlaza el atributo HTML disabled al prop disabled. Cuando es true, el navegador bloquea el clic nativamente.
<code>@click="\$emit('click')"</code>	Cuando el botón se presiona (y no está disabled), emite el evento 'click' hacia App.vue que ejecuta generarFactura().
<code>{{ label }}</code>	El texto del botón es configurable — puede decir 'Facturar' o cualquier otro texto que App.vue pase.

Cómo funciona el Script:

Código	Explicación
<code>disabled: { type: Boolean }</code>	Prop que controla si el botón está activo. App.vue lo enlaza al computed isValid.

Cómo funciona el CSS:

Regla CSS	Explicación
<code>border-radius: var(--button-radius)</code>	Usa la variable --button-radius (30px) del archivo tokens.css para el efecto píldora.
<code>transition: background 0.2s</code>	Animación suave al cambiar de color en hover y al escalar en click.
<code>:disabled { opacity: 0.6 }</code>	Estado visual deshabilitado: el botón se vuelve gris semitransparente y el cursor cambia a 'prohibido'.

ToggleSwitch.vue — Interruptor deslizante

Propósito: Switch que alterna entre activado/desactivado. El checkbox real está invisible; el efecto visual se logra puramente con CSS usando el selector para cambiar el estilo del slider cuando el checkbox está marcado.

Cómo funciona el Template:

Código / Elemento	Explicación
<code>:checked="modelValue"</code>	Muestra el estado actual (true/false) en el checkbox del DOM.
<code>@change="\$emit('update:modelValue', ...)"</code>	Cuando el usuario hace clic, emite el nuevo estado booleano. \$event.target.checked es true o false.
<code></code>	Este elemento vacío es el track visual y el círculo deslizante, dibujados completamente con CSS.

Cómo funciona el Script:

Código	Explicación
<code>modelValue: { type: Boolean }</code>	Vue sabe que este prop debe ser boolean, no string. Si se pasa un tipo incorrecto, Vue muestra advertencia en consola.

Cómo funciona el CSS:

Regla CSS	Explicación
<code>.toggle__input { opacity: 0; width: 0; height: 0 }</code>	Oculto el checkbox real sin usar <code>display:none</code> , para que siga siendo funcional y accesible.
<code>.toggle__slider::before</code>	El pseudo-elemento <code>::before</code> crea el círculo blanco que desliza. No se necesita HTML extra.
<code>.toggle__input:checked ~ .toggle__slider</code>	El selector <code>~</code> selecciona el hermano siguiente. Cuando el checkbox se marca, el track cambia a azul.
<code>transform: translateX(20px)</code>	El círculo se desplaza 20px a la derecha con una transición suave de 0.25s.

5. DropdownSelect.vue — Menú desplegable

Propósito: Selector estilizado con flecha personalizada. Recibe el array de opciones como prop para ser completamente reutilizable — el mismo componente sirve para CFDI (24 opciones) y Régimen Fiscal (26 opciones).

Cómo funciona el Template:

Código / Elemento	Explicación
<code>v-for="opt in options"</code>	Vue itera el array y crea un <code><option></code> por cada entrada. No es necesario escribir las opciones a mano en el HTML.
<code>:disabled="opt.value === ''"</code>	La primera opción (placeholder) tiene value vacío, así que queda deshabilitada.
<code>:value="modelValue" en <select></code>	Sincroniza el select con el estado de App.vue. Vue selecciona automáticamente la opción correcta.
<code>@change="\$emit('update:modelValue', ...)"</code>	Al cambiar la selección, emite el value de la opción elegida.
<code>▼</code>	Ícono personalizado superpuesto. <code>pointer-events: none</code> permite que los clics pasen al select.



Cómo funciona el Script:

Código	Explicación
<pre>options: { type: Array, default: () => [] }</pre>	Default es una función que retorna array vacío — obligatorio en Vue para tipos Array/Object.

Cómo funciona el CSS:

Regla CSS	Explicación
<code>appearance: none</code>	Elimina la flecha nativa del navegador para mostrar la nuestra propia.
<code>padding: 0 40px 0 16px</code>	El padding derecho de 40px deja espacio para que la flecha no tape el texto de la opción.

6. AddSectionButton.vue — Botón agregar sección

Propósito: Botón circular azul con ícono '+' creado con SVG inline. Al hacer clic, App.vue ejecuta la lógica de dos fases: el primer clic oculta el Monto azul y activa el modo multi-sección; los siguientes clics agregan secciones adicionales.

Cómo funciona el Template:

Código / Elemento	Explicación
<code><svg></code> con dos <code><line></code>	El ícono '+' se dibuja con SVG puro — dos líneas perpendiculares. No depende de ninguna librería de íconos.
<code>@click="\$emit('click')"</code>	Delega la lógica al padre. El componente no sabe qué hace el clic — solo avisa. App.vue decide qué pasa.

Cómo funciona el Script:

Código	Explicación
<code>emits: ['click']</code>	Declarar el evento explícitamente en Vue 3 evita que el clic nativo del botón se propague dos veces.

Cómo funciona el CSS:

Regla CSS	Explicación
<code>justify-content: flex-end</code>	Alinea el botón y su etiqueta a la derecha de la columna, tal como aparece en el diseño Figma.
<code>border-radius: 50%</code>	Convierte el cuadrado de 36x36px en un círculo perfecto.

7. ExtraSection.vue — Sección fiscal adicional

Propósito: Componente reutilizable generado dinámicamente por v-for en App.vue. Cada instancia tiene su propio objeto de datos, color alternado calculado con módulo matemático, y número de sección calculado con computed.

Cómo funciona el Template:

Código / Elemento	Explicación
<code>:class="`extra-section__divide r--\${color}`"</code>	Aplica dinámicamente la clase de color al divisor y al título de sección.
Razón Fiscal adicional <code>#{{ displayIndex }}</code>	Muestra el número de sección calculado. Index 0 → #2, index 1 → #3, etc.
<code>@update:model-value="update('correo', \$event)"</code>	Cada InputField hija llama al método <code>update()</code> con el nombre del campo y el nuevo valor.
<code>@open-modal="\$emit('open-modal', \$event)"</code>	Burbujeo de eventos: ExtraSection pasa el evento <code>open-modal</code> de InputField hacia App.vue.

Cómo funciona el Script:

Código	Explicación
<code>displayIndex() { return this.index + 2 }</code>	Computed que convierte el índice base-0 en el número de sección visible. Index 0 = Sección 2.
<code>color() { return index % 2 === 0 ? 'yellow' : 'green' }</code>	El operador <code>%</code> (módulo) alterna entre <code>yellow</code> y <code>green</code> según si el índice es par o impar.
<code>update(field, value) { ...this.modelValue, [field]: value }</code>	Spread operator: copia todos los campos y sobrescribe solo el campo modificado.

Cómo funciona el CSS:

Regla CSS	Explicación
<code>animation: slideIn 0.3s ease</code>	Cada nueva sección aparece deslizándose desde arriba con opacidad 0 a 1 en 0.3 segundos.
<code>border-top: 2px dashed</code>	El divisor es una línea punteada cuyo color cambia por clase (<code>green</code> o <code>yellow</code>).

8. HelpModal.vue — Modal de ayuda con imagen

Propósito: Ventana emergente de pantalla completa que muestra la fotografía real del ticket o de la Constancia Fiscal. El contenido se calcula automáticamente con computed properties según el prop type.

Cómo funciona el Template:

Código / Elemento	Explicación
<code>v-if="isOpen"</code>	Cuando <code>isOpen</code> es false, el modal es completamente eliminado del DOM — no solo ocultado.
<code>@click.self="\$emit('close')"</code>	El modificador <code>.self</code> garantiza que el evento solo se dispare si el clic fue en el overlay, no en el modal interior.
<code>v-html="subtitle"</code>	Renderiza HTML real (con etiquetas <code></code>) dentro del párrafo para resaltar los nombres de campo en negritas.

Cómo funciona el Script:

Código	Explicación
<code>isTicket() { return this.type === 'ticket' }</code>	Computed base que los demás usan como condición.
<code>title(), subtitle(), image()</code>	Tres computed que usan <code>isTicket</code> . Vue los recalcula automáticamente si cambia el prop type.

Cómo funciona el CSS:

Regla CSS	Explicación
<code>position: fixed; inset: 0</code>	<code>inset: 0</code> equivale a <code>top/right/bottom/left: 0</code> — cubre toda la pantalla. <code>fixed</code> lo mantiene fijo al hacer scroll.
<code>z-index: 200</code>	Garantiza que el modal aparezca por encima de todos los demás elementos de la página.
<code>animation: popIn 0.22s ease</code>	El modal aparece con un efecto de escala de 0.9 a 1.0, dando sensación de pop natural.
<code>max-height: 88vh; overflow-y: auto</code>	En pantallas pequeñas, el modal tiene scroll si el contenido es mayor al 88% de la altura de pantalla.

9. SuccessModal.vue — Modal de confirmación

Propósito: Modal que confirma la generación de la factura. Recibe el array de filas ya construido desde `App.vue` y lo renderiza en una tabla. No contiene lógica propia — toda la construcción de datos ocurre en el método `buildRows()` de `App.vue`.



Cómo funciona el Template:

Código / Elemento	Explicación
<code>v-if="isOpen"</code>	El modal desaparece completamente del DOM cuando no está visible, igual que HelpModal.
<code>v-for="row in rows"</code>	Itera el array rows y crea una fila <tr> por cada campo del formulario.
<code>:key="row.label"</code>	Vue necesita una key única por elemento en v-for para hacer actualizaciones eficientes del DOM.
<code>{{ row.value '-' }}</code>	Si el valor está vacío, muestra un guión largo. El operador evalúa el string vacío como falsy.
<code>:class="row.value ? '' : 'is-empty'"</code>	Aplica clase 'is-empty' (texto gris) solo a las celdas vacías.

Cómo funciona el Script:

Código	Explicación
<code>rows: { type: Array, default: () => [] }</code>	Recibe el array completo pre-construido por buildRows() en App.vue. No sabe cómo se construyeron los datos.

Cómo funciona el CSS:

Regla CSS	Explicación
<code>.modal--success { text-align: center }</code>	El ícono y el título quedan centrados, mientras la tabla se alinea a la izquierda.
<code>border-collapse: collapse</code>	Elimina el espacio doble entre bordes de celdas adyacentes de la tabla.
<code>border-bottom: 1px solid #eee</code>	Línea separadora sutil entre filas, sin bordes laterales para apariencia más limpia.

Pruebas Unitarias — Vue Test Utils

Se implementaron 30 pruebas unitarias usando Vitest + @vue/test-utils + jsdom. Cada prueba monta el componente de forma aislada, simula interacciones del usuario y verifica que el comportamiento sea el esperado.

ActionButton
1. renders the label prop correctly
2. emits click event when clicked and not disabled
3. does not emit click when disabled
4. button has disabled attribute when prop is true
InputField
1. renders the label correctly
2. applies blue class when color is blue
3. applies green class when color is green
4. shows help button when help prop is set
5. hides help button when help prop is empty
6. emits update:modelValue when user types
ToggleSwitch
1. renders the label correctly
2. starts unchecked by default
3. emits update:modelValue with true when turned on
4. emits update:modelValue with false when turned off
5. checked getter returns correct state
DropdownSelect
1. renders all options
2. first option is disabled (placeholder)
3. emits update:modelValue when option is selected
4. select shows correct value when modelValue prop is set
HelpModal
1. is hidden when isOpen is false
2. is visible when isOpen is true
3. shows ticket title when type is ticket
4. shows constancia title when type is constancia
5. emits close when × button is clicked
6. emits close when backdrop is clicked
SuccessModal
1. is hidden when isOpen is false
2. is visible when isOpen is true
3. renders all rows in the table
4. shows — for empty values
5. emits close when × is clicked

```
PS C:\Users\RAM\Downloads\Componentes\V2\files 5 - copia> npm test
```

```
✓ src/tests/SuccessModal.test.js (5)
✓ src/tests/ActionButton.test.js (4)
```

```
Test Files 6 passed (6)
```

```
Tests 30 passed (30)
```

```
Start at 22:14:48
```

```
Duration 1.25s (transform 270ms, setup 0ms, collect 952ms, tests 193ms, environment 4.75s, prepare 649ms)
```